# Generative Modeling via Flow Matching: A Comprehensive Mathematical and Algorithmic Analysis

Detailed Technical Report

March 28, 2026

## Contents


**1 Introduction to Generative Modeling and Continuous Flows** **2**

**2 Mathematical Foundations of Flow Matching** **2**

2.1 Probability Paths and Vector Fields . . . . . 2

2.2 Conditional Flow Matching (CFM) . . . . . 2

2.3 Deriving the Optimal Transport (Linear) Probability Path . . . . . 3

**3 The Critical Role of Residual Connections** **4**

3.1 The Vanishing Gradient Problem . . . . . 4

3.2 The Residual Solution . . . . . 4

3.3 Residuals and ODE Interpretation . . . . . 5

**4 Exhaustive Line-by-Line Code Analysis** **6**

4.1 Data Generation: `sample.py` . . . . . 6

4.2 Model Architecture: `fm.py` . . . . . 8

4.3 Training and Inference: `train.py` . . . . . 10

**5 Conclusion** **12**

# 1 Introduction to Generative Modeling and Continuous Flows

Generative modeling aims to learn the underlying probability distribution of a given dataset, enabling the generation of novel samples that resemble the original data. Among the various paradigms—such as Generative Adversarial Networks (GANs), Variational Autoencoders (VAEs), and Diffusion Models—Continuous Normalizing Flows (CNFs) have emerged as a mathematically elegant framework.

CNFs model the transformation of a simple base distribution (usually a standard Gaussian) into a complex target distribution (the data) via an Ordinary Differential Equation (ODE). Flow Matching (FM) is a recent, highly efficient framework for training CNFs. Instead of relying on expensive numerical ODE solvers during training or dealing with the complex score-matching objective of diffusion models, Flow Matching directly regresses a neural network onto constructed target vector fields.

This report provides a thesis-level exposition of Flow Matching, focusing on the mathematical derivations of the probability paths, the necessity of residual connections in the architecture, and an exhaustive, line-by-line analysis of the provided implementation.

# 2 Mathematical Foundations of Flow Matching

## 2.1 Probability Paths and Vector Fields

Let $p_0(x)$ be a simple prior distribution, typically standard normal $\mathcal{N}(0, I)$, defined on $\mathbb{R}^d$. Let $p_1(x)$ be the complex empirical data distribution. We want to construct a time-dependent probability density path $p_t(x)$ for $t \in [0, 1]$ such that it smoothly interpolates between $p_0$ and $p_1$.

By the continuity equation, the evolution of this probability density is governed by a time-dependent vector field $v_t : \mathbb{R}^d \to \mathbb{R}^d$:

$$\frac{\partial}{\partial t} p_t(x) = -\nabla \cdot (p_t(x) v_t(x)) \tag{1}$$

If we can construct a target vector field $v_t(x)$ that generates the path $p_t$, we can train a neural network $v_\theta(x, t)$ to approximate it using a simple Mean Squared Error (MSE) objective:

$$\mathcal{L}_{FM}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0,1], x \sim p_t(x)} \left[ \| v_\theta(x, t) - v_t(x) \|^2 \right] \tag{2}$$

However, computing the marginal vector field $v_t(x)$ directly is generally intractable because it requires marginalizing over the entire dataset.

## 2.2 Conditional Flow Matching (CFM)

To make training tractable, Flow Matching introduces conditional probability paths. We condition on individual data points $x_1 \sim p_1(x)$. Let $p_t(x|x_1)$ be a conditional probability

path that transforms $p_0(x)$ into a Dirac delta centered at $x_1$, i.e., $p_1(x|x_1) = \delta(x - x_1)$.

Corresponding to this conditional path is a conditional vector field $v_t(x|x_1)$. The Conditional Flow Matching objective is:

$$\mathcal{L}_{CFM}(\theta) = \mathbb{E}_{t,q(x_1),p_t(x|x_1)} \left[ \|v_\theta(x,t) - v_t(x|x_1)\|^2 \right] \tag{3}$$

It has been mathematically proven that minimizing $\mathcal{L}_{CFM}(\theta)$ is equivalent to minimizing $\mathcal{L}_{FM}(\theta)$ up to a constant independent of $\theta$.

## 2.3 Deriving the Optimal Transport (Linear) Probability Path

A straightforward and highly effective choice for the conditional path is the Optimal Transport (OT) displacement interpolation. We define the conditional path as a Gaussian with a time-varying mean and a fixed (or shrinking) variance. For simplicity, and as implemented in the code, we consider the deterministic, zero-variance limit (or interpolation of points):

$$x_t = \psi_t(x_0, x_1) = (1 - t)x_0 + tx_1 \tag{4}$$

where $x_0 \sim p_0$ (noise) and $x_1 \sim p_1$ (data).

To find the target conditional vector field $v_t(x_t|x_1)$ that generates this path, we simply take the time derivative of the position $x_t$ with respect to time $t$:

$$\frac{d}{dt}x_t = \frac{d}{dt}\left[(1 - t)x_0 + tx_1\right] \tag{5}$$

$$\frac{d}{dt}x_t = -x_0 + x_1 = x_1 - x_0 \tag{6}$$

Thus, the target velocity is constant along the trajectory from $x_0$ to $x_1$. The objective function simplifies immensely to:

$$\mathcal{L}(\theta) = \mathbb{E}_{t\sim\mathcal{U}(0,1),x_0\sim p_0,x_1\sim p_1} \left[ \|v_\theta(x_t,t) - (x_1 - x_0)\|^2 \right] \tag{7}$$

This exact equation is the theoretical backbone of the training loop implemented in `train.py`.

# 3 The Critical Role of Residual Connections

In earlier versions of your code, the `Block` class in the MLP did not have a residual connection, which caused the network's predictions to become unstable, leading to points flying out of bounds during ODE integration. Let us rigorously analyze why the residual connection $y = x + f(x)$ is vital.

## 3.1 The Vanishing Gradient Problem

Consider a deep Multi-Layer Perceptron (MLP) without residual connections. Let the input to the $l$-th layer be $h_l$. The forward pass is:

$$h_{l+1} = \sigma(W_l h_l + b_l) \tag{8}$$

where $\sigma$ is an activation function (like ReLU). During backpropagation, we compute the gradient of the loss $\mathcal{L}$ with respect to the input $h_l$:

$$\frac{\partial \mathcal{L}}{\partial h_l} = \frac{\partial \mathcal{L}}{\partial h_{l+1}} \frac{\partial h_{l+1}}{\partial h_l} = \frac{\partial \mathcal{L}}{\partial h_{l+1}} \left( W_l^T \sigma'(W_l h_l + b_l) \right) \tag{9}$$

If the network has $L$ layers, the gradient with respect to the first layer involves the product of $L$ Jacobian matrices:

$$\frac{\partial \mathcal{L}}{\partial h_1} = \frac{\partial \mathcal{L}}{\partial h_L} \prod_{i=1}^{L-1} W_i^T \sigma'(z_i) \tag{10}$$

If the singular values of the weight matrices $W_i$ are less than 1, or if the activation gradients $\sigma'$ are small, this product exponentially decays to zero. This is the vanishing gradient problem. The network cannot learn.

## 3.2 The Residual Solution

Now, introduce a skip connection:

$$h_{l+1} = h_l + f_l(h_l) \tag{11}$$

where $f_l$ represents the layer transformations $\sigma(W_l h_l + b_l)$. The gradient now becomes:

$$\frac{\partial \mathcal{L}}{\partial h_l} = \frac{\partial \mathcal{L}}{\partial h_{l+1}} \frac{\partial}{\partial h_l}(h_l + f_l(h_l)) = \frac{\partial \mathcal{L}}{\partial h_{l+1}} \left( I + \frac{\partial f_l(h_l)}{\partial h_l} \right) \tag{12}$$

Expanding this over $L$ layers gives:

$$\frac{\partial \mathcal{L}}{\partial h_1} = \frac{\partial \mathcal{L}}{\partial h_L} \left( I + \sum_{i=1}^{L-1} \frac{\partial f_i}{\partial h_i} + \dots \right) \tag{13}$$

The identity matrix $I$ provides a direct, unattenuated "highway" for the gradient to flow all the way back to the initial layers. The term $\frac{\partial \mathcal{L}}{\partial h_L}$ directly reaches $h_1$ without being multiplied by potentially small layer Jacobian matrices.

## 3.3 Residuals and ODE Interpretation

Furthermore, there is a profound connection between residual networks and Euler integration of ODEs. An ODE $dx/dt = f(x, t)$ approximated by the Euler method with step size $\Delta t = 1$ is:

$$x_{t+1} = x_t + f(x_t, t) \tag{14}$$

A residual block $h_{l+1} = h_l + f(h_l)$ is mathematically equivalent to taking one Euler step along an abstract vector field $f$ defined by the neural network's layers. In Flow Matching, our neural network models a smooth velocity field. By using residual connections, the architecture itself is biased towards learning smooth, incremental changes (identity mapping plus a small perturbation), which perfectly aligns with the continuous, smooth nature of the vector fields we are trying to learn. Without residuals, the network struggles to pass the unperturbed spatial information $x$ forward, leading to chaotic, extreme velocity predictions that cause the ODE integrator to explode out of bounds $[-4, 4]$.

# 4 Exhaustive Line-by-Line Code Analysis

## 4.1 Data Generation: `sample.py`

This script generates a 2D checkerboard distribution. It relies on rejection sampling to filter uniformly distributed noise into a structured pattern.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 N = 1000
5 x_min, x_max = -4, 4
6 y_min, y_max = -4, 4
7 resolution = 100
```

**Lines 1-7:** Standard imports and parameter definitions. We aim to sample $N = 1000$ points within the spatial domain $[-4, 4] \times [-4, 4]$.

```
1 x = np.linspace(x_min, x_max, resolution)
2 y = np.linspace(y_min, y_max, resolution)
3 X, Y = np.meshgrid(x, y)
```

**Lines 9-11:** Creates a discretized 2D grid using `np.meshgrid`. While defined, `X` and `Y` are surprisingly unused in the actual sampling logic below, though they are often used for evaluating density functions over a grid.

```
1 length = 4
2 checkboard = np.indices((length, length)).sum(axis=0) % 2
```

**Lines 13-14:** This constructs the logical mask for the checkerboard. `np.indices((4, 4))` returns two $4 \times 4$ arrays containing the row and column indices.

- Array 1 (rows): $i_{r,c} = r$
- Array 2 (cols): $j_{r,c} = c$

Summing them along `axis=0` gives $M_{r,c} = r + c$. Taking the modulo 2 (`% 2`) yields 1 if $r + c$ is odd and 0 if even. This perfectly defines a $4 \times 4$ alternating binary matrix.

```
1 sampled_points = []
2 while len(sampled_points) < N:
3     x_sample = np.random.uniform(x_min, x_max)
4     y_sample = np.random.uniform(y_min, y_max)
```

**Lines 16-20:** The Rejection Sampling loop begins. It continues until exactly 1000 points are collected. We propose candidate points $(x, y) \sim \mathcal{U}(-4, 4)$.

```
1     i = int((x_sample - x_min) / (x_max - x_min) * length)
2     j = int((y_sample - y_min) / (y_max - y_min) * length)
```

**Lines 22-23:** Discretization. We map the continuous coordinate $x \in [-4, 4)$ to an integer index $i \in \{0, 1, 2, 3\}$. Derivation: 1. Translate to origin: $x - (-4) = x + 4 \in [0, 8)$. 2.

Normalize to $[0, 1)$: $\frac{x+4}{8}$. 3. Scale to grid size: $\frac{x+4}{8} \times 4 \in [0, 4)$. 4. Floor to integer using `int()`: yields index $i$.

```
    if checkboard[j, i] == 1:
        sampled_points.append((x_sample, y_sample))

sampled_points = np.array(sampled_points)
```

**Lines 25-29:** The acceptance criterion. If the proposed point falls into a grid cell corresponding to a '1' in the checkerboard mask, it is accepted and appended. Finally, the list is converted to a NumPy array of shape $(1000, 2)$.

## 4.2 Model Architecture: `fm.py`

This file defines the neural network responsible for modeling the continuous vector field $v_\theta(x, t)$.

```
1  import math
2  import torch
3  import torch.nn as nn
4
5  class Block(nn.Module):
6      def __init__(self, channels=512) -> None:
7          super().__init__()
8          self.ff = nn.Linear(channels, channels)
9          self.act = nn.ReLU()
10
11     def forward(self, x):
12         return x + self.act(self.ff(x))
```

**Lines 1-12 (`Block`):** This defines a single Residual Block.

- `self.ff` applies an affine transformation $Wx + b$.
- `self.act` applies the non-linear Rectified Linear Unit, $\text{ReLU}(z) = \max(0, z)$.
- **Line 12:** The explicitly added residual connection $y = x + \text{ReLU}(Wx + b)$. As derived in Section 3, this is paramount for gradient flow and ODE stability.

```
1  class MLP(nn.Module):
2      def __init__(self, channels_data=2, layers=5, channels=512, channels_t
           =512) -> None:
3          super().__init__()
4          self.channels_t = channels_t
5          self.in_projection = nn.Linear(channels_data, channels)
6          self.t_projection = nn.Linear(channels_t, channels)
7          self.blocks = nn.Sequential(*[
8                  Block(channels) for _ in range(layers)
9              ])
10         self.out_projection = nn.Linear(channels, channels_data)
```

**Lines 14-23 (`MLP` Initialization):** Defines the main Multi-Layer Perceptron.

- `in_projection`: Maps the 2D input coordinates ($x \in \mathbb{R}^2$) up to the hidden dimension $d = 512$.
- `t_projection`: A linear layer applied to the sinusoidal time embeddings.
- `blocks`: A sequential chain of 5 residual `Block` layers.
- `out_projection`: Maps the 512-dimensional hidden state back down to the 2D output representing the velocity vector $v \in \mathbb{R}^2$.

```
    def gen_t_embedding(self, t, max_positions=10000):
        t = t * max_positions
        half_dim = self.channels_t // 2
        emb = math.log(max_positions) / (half_dim - 1)
        emb = torch.arange(half_dim, device=t.device).float().mul(-emb).exp
            ()
        emb = t[:, None] * emb[None, :]
        emb = torch.cat([emb.sin(), emb.cos()], dim=1)
        if self.channels_t % 2 == 1:
            emb = nn.functional.pad(emb, (0, 1), mode='constant')
        return emb
```

**Lines 25-35 (Sinusoidal Time Embeddings):** Neural networks inherently struggle to learn high-frequency variations from low-dimensional inputs (like a scalar time $t$). Positional embeddings map $t$ to a high-dimensional hypersphere, solving this. Mathematical derivation of the code: 1. `half_dim` $= d/2 = 256$. 2. We want frequencies scaling geometrically from 1 to 10000. The logarithmic step is: $\Delta = \frac{\ln(10000)}{256-1}$ (Line 28). 3. The frequencies are $f_i = \exp(-i \cdot \Delta)$ for $i \in \{0, \dots, 255\}$ (Line 29). This evaluates to $10000^{-i/(d/2-1)}$. 4. We multiply time $t$ by these frequencies (Line 30): $\theta_{t,i} = t \cdot 10000 \cdot f_i$. 5. We compute sine and cosine phases (Line 31) and concatenate them: $\text{emb}_t = [\sin(\theta_{t,0}), \dots, \sin(\theta_{t,255}), \cos(\theta_{t,0}), \dots, \cos(\theta_{t,255})]$. This exact formulation is adapted from the original Transformer paper ("Attention Is All You Need").

```
    def forward(self, x, t):
        x = self.in_projection(x)
        t = self.gen_t_embedding(t)
        t = self.t_projection(t)
        x = x + t
        x = self.blocks(x)
        x = self.out_projection(x)
        return x
```

**Lines 37-44 (Forward Pass):** 1. `x` (shape `N, 2`) is projected to shape `N, 512`. 2. Time $t$ (shape `N`) is converted to embeddings of shape `N, 512`. 3. The time embeddings are linearly projected. 4. **Conditioning:** Time is injected into the state via addition: $h = x + t$. This tells the network where it is in the ODE integration process. 5. The state passes through the residual blocks. 6. The state is projected back to shape `N, 2` using `out_projection`. (This was previously bugged and erroneously used `t_projection`, causing shape mismatch errors).

## 4.3 Training and Inference: `train.py`

This script executes the Flow Matching theory, sampling from probability paths and regressing the neural vector field.

```
1  import matplotlib.pyplot as plt
2  from matplotlib.animation import FuncAnimation
3  import torch
4  from torch._C import device
5  import torch.optim as optim
6  import tqdm
7  from sample import sampled_points
8  from fm import MLP
9
10 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
11
12 model = MLP(layers=5, channels=512).to(device)
13 optimizer = optim.AdamW(model.parameters(), lr=1e-4)
```

**Lines 1-13:** Setup. Imports necessary libraries. Detects hardware acceleration (CUDA). Initializes the `MLP` model and moves it to the appropriate device. Initializes the AdamW optimizer with a learning rate of $10^{-4}$.

```
1 data = torch.Tensor(sampled_points)
2 training_steps = 100_000
3 batch_size = 512
4 pbar = tqdm.tqdm(range(training_steps))
5 losses = []
```

**Lines 15-19:** Converts the pre-generated checkerboard points into a PyTorch Tensor. Defines hyperparameters: 100,000 optimization steps and a batch size of 512. `tqdm` is used for a progress bar.

```
1 for i in pbar:
2     x1 = data[torch.randint(data.size(0), (batch_size,))]
3     x0 = torch.randn_like(x1)
4     target = (x1 - x0).to(device)
5     t = torch.rand(x1.size(0))
6     xt = (1 - t[:, None]) * x0 + t[:, None] * x1
```

**Lines 20-26 (The Core Flow Matching Logic):**

- **Line 21:** Uniformly samples indices to select a batch of target data points $x_1 \sim p_1(x)$. Shape: $(512, 2)$.
- **Line 22:** Samples source noise $x_0 \sim \mathcal{N}(0, I)$. This replaces the buggy `rand_like` (Uniform noise) with `randn_like` (Gaussian noise), which is the standard assumption for the prior $p_0$ in generative flows.
- **Line 23:** Calculates the true, constant velocity vector generating the path: $v_t = x_1 - x_0$. This is our regression target.

- **Line 24:** Samples uniform time steps $t \sim \mathcal{U}(0, 1)$ for each element in the batch. Shape: $(512, )$.
- **Line 25:** Samples intermediate points along the linear probability path $x_t = (1 - t)x_0 + tx_1$. `t[:, None]` broadcasts the scalar time to match the 2D spatial dimensions.

```
    pred = model(xt.to(device), t.to(device))
    loss = ((target - pred)**2).mean()
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
    pbar.set_postfix(loss=loss.item())
    losses.append(loss.item())
```

**Lines 27-33:** 1. The neural network predicts the velocity $v_\theta(x_t, t)$. 2. The MSE loss is computed between the predicted velocity and the true target velocity. This directly implements the equation $\mathcal{L}(\theta) = \mathbb{E}[\|v_\theta - (x_1 - x_0)\|^2]$. 3. Backpropagation (`backward()`) computes gradients. 4. `optimizer.step()` updates the network weights. 5. `zero_grad()` clears the gradients for the next iteration.

```
# Inference
trajectory = []

xt = torch.randn(1000, 2).to(device)
steps = 200  # reduce for smoother video

for t in torch.linspace(0, 1, steps):
    trajectory.append(xt.detach().cpu().numpy())

    pred = model(xt, t.expand(xt.size(0)).to(device))
    xt = xt + (1 / steps) * pred
```

**Lines 43-52 (ODE Integration):** Once trained, we generate data by simulating the ODE $dx/dt = v_\theta(x, t)$. 1. **Line 46:** Sample 1000 points from the base distribution $p_0 \sim \mathcal{N}(0, I)$. 2. **Line 49:** Iterate time $t$ from 0 to 1 in 200 discrete steps. $\Delta t = 1/200$. 3. **Line 52:** Predict the velocity field at the current time and state. `t.expand` ensures the scalar time step is broadcasted to the batch dimension. 4. **Line 53:** The Euler Method step: $x_{new} = x_{old} + \Delta t \cdot v_\theta(x_{old}, t)$. This moves the points incrementally along the learned vector field until they approximate the target checkerboard distribution at $t = 1$. The addition of device movement (`.to(device)`) for `xt` and `t` in inference fixed the device mismatch errors encountered during testing.

```
fig, axes = plt.subplots(1, 2, figsize=(10, 5))
...
anim = FuncAnimation(fig, update, frames=len(trajectory), interval=50)
plt.tight_layout()
plt.show()
```

**Lines 56-74:** Standard Matplotlib animation boilerplate. It iterates over the recorded `trajectory` list and dynamically updates the scatter plot offsets, allowing visualization of the continuous normalization flow mapping standard Gaussian noise into the complex checkerboard geometry.

# 5 Conclusion

This exhaustive analysis details the transition from the theoretical continuous probability formulation of Flow Matching down to the practical implementation in PyTorch. We derived the linear probability path, proved the necessity of the residual connections to prevent vanishing gradients and maintain ODE integration stability, and dissected the exact mathematics behind time embeddings and numerical ODE solving. The resulting code now acts as a mathematically rigorous, numerically stable generative model.